

Hybrid Search & Retrieval

Instructor: Your Name

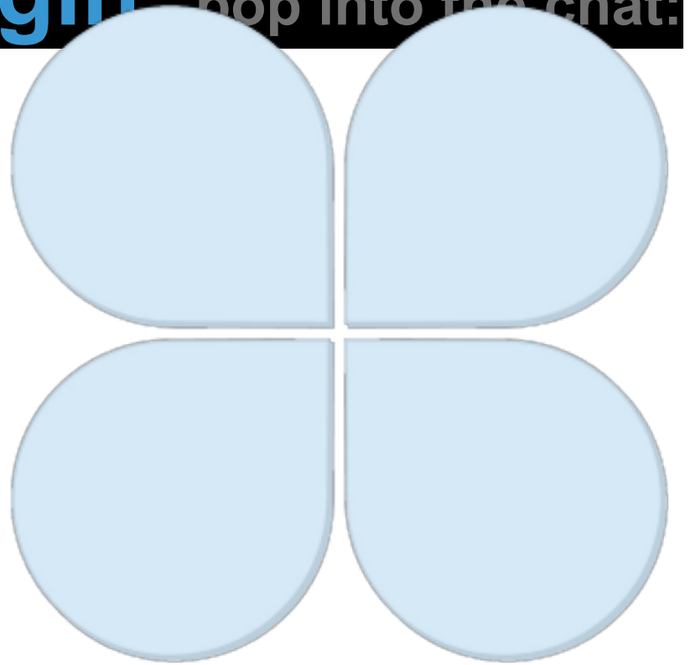
{ik} INTERVIEW KICKSTART



Julie Wilkins

- Senior Product Manager, Amazon
- MBA – Temple University
- BA – Louisiana State University

Welcome to today's class, before we begin pop into the chat:



Your Name Your Role

Your Company Your Location



Optimize Your Experience

Interact with Instructors

Join Sunday live class to ask questions and get clarifications.

Engage with Assessments

Complete assignments and MCQs. Get feedback during Thursday review sessions.

Utilize Your Resources

Make the most of all available resources to maximize your learning experience.

Are you using all these IK support features?

In-level Resources

⚠ Contains all the learning resources - videos, MCQs, assignments.

⚠ [redacted]

Sessions on Wednesday to clarify all your queries.

⚠ [redacted]

TAs help answer queries in the live class.

⚠ [redacted]Optional videos for additional reference.

⚠

[redacted]

For peer learning and collaboration.

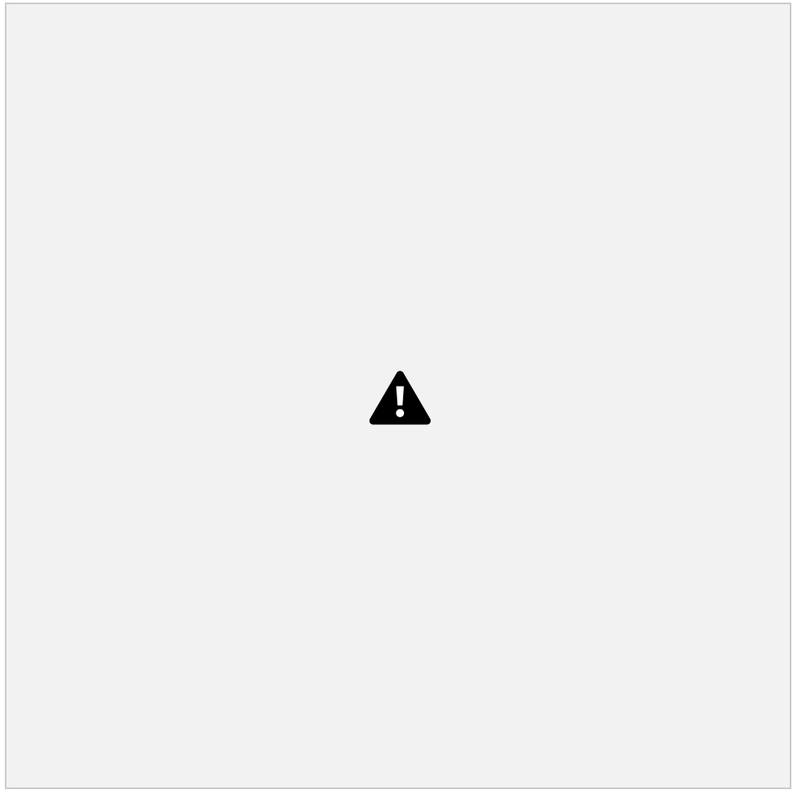
⚠

[redacted]

Raise them any time you want any

clarifications/tech concerns, etc on UpLevel.

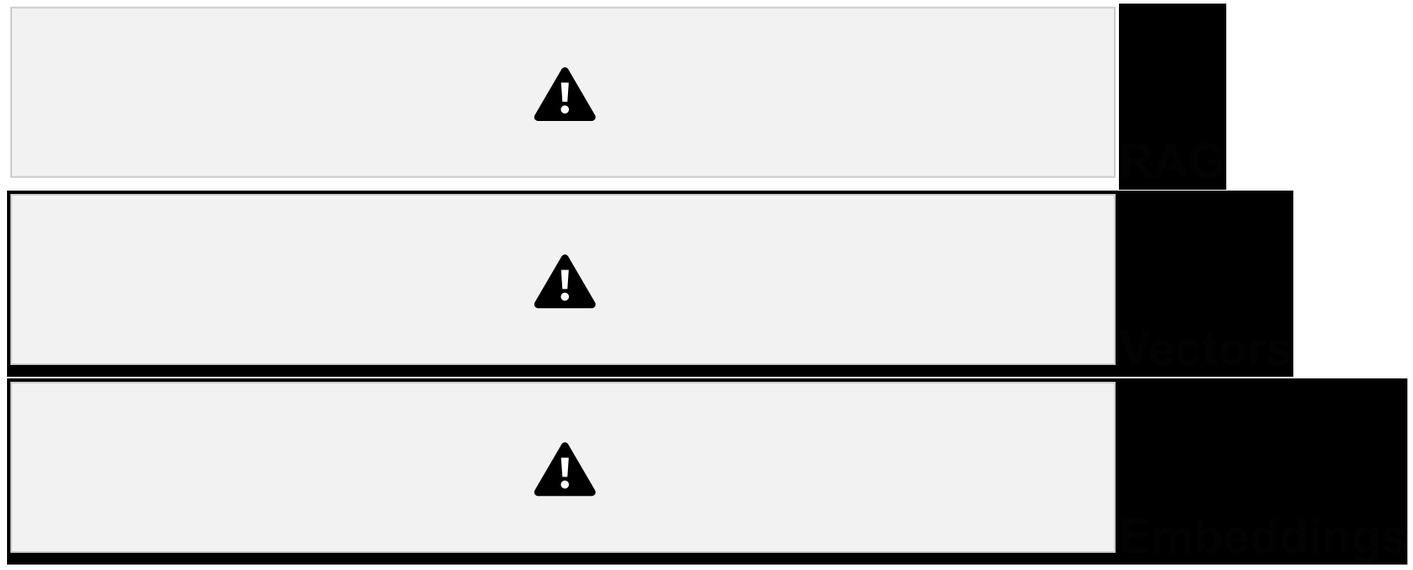
Learning Objectives

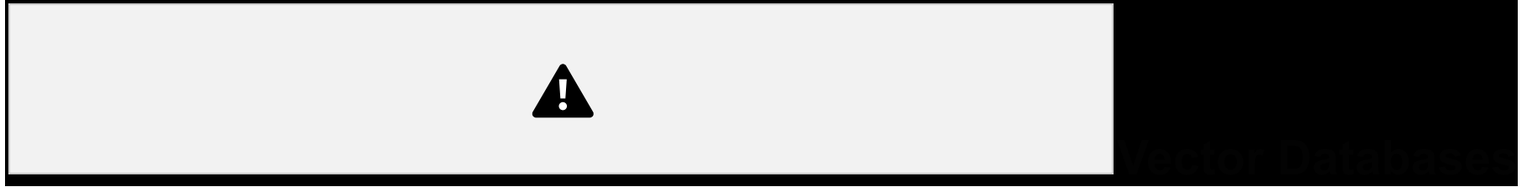


[redacted]

Before We Begin

We assume that from previous lectures you already have understanding of:



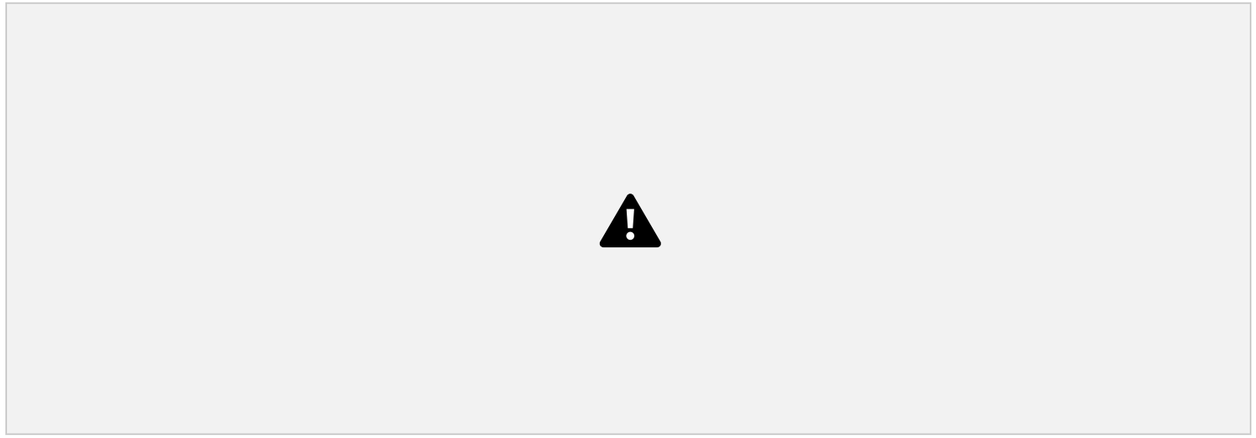


Recap

Let's revisit the foundational concepts before we dive in.

Vector Embeddings

Vector embeddings are numerical representations of data points that express different types of data, including non-mathematical data such as words or images, as an array of numbers that machine learning (ML) models can process.

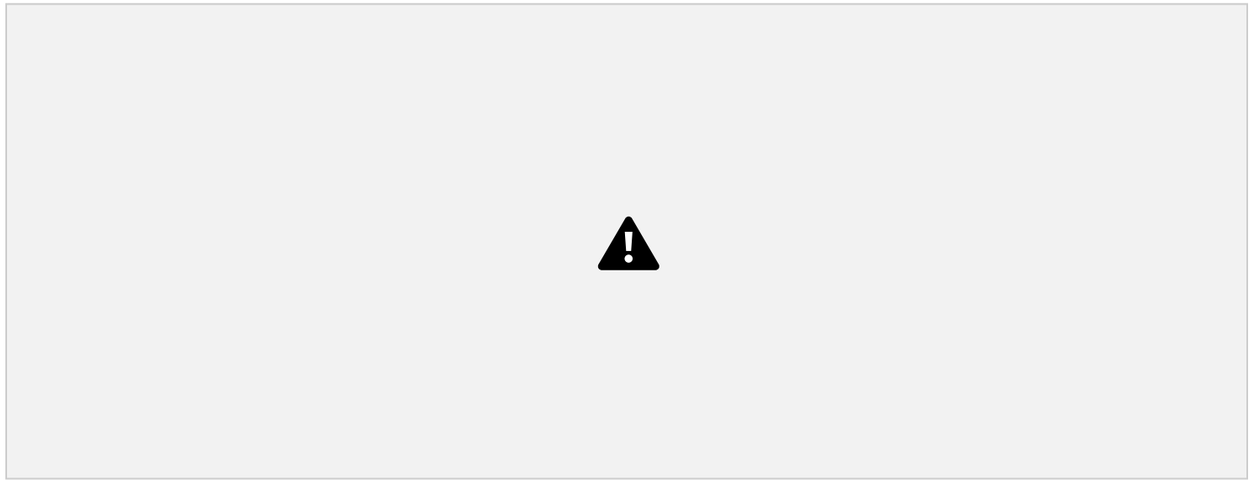


For a detailed walkthrough of these concepts, please refer to the previous class recordings on RAG.

RECAP

Vector Databases

A vector database indexes and stores vector embeddings for fast retrieval and similarity search, with capabilities like CRUD operations, metadata filtering, horizontal scaling, and serverless.



For a detailed walkthrough of these concepts, please refer to the previous class recordings on RAG.

Vector Index vs Vector Databases

⚠ **Vector Index (e.g. FAISS)**

[redacted]

- [ ] [redacted]
- [ ] [redacted]
- [ ] [redacted]

base based on user queries.

Relevant information is identified and extracted from the knowledge

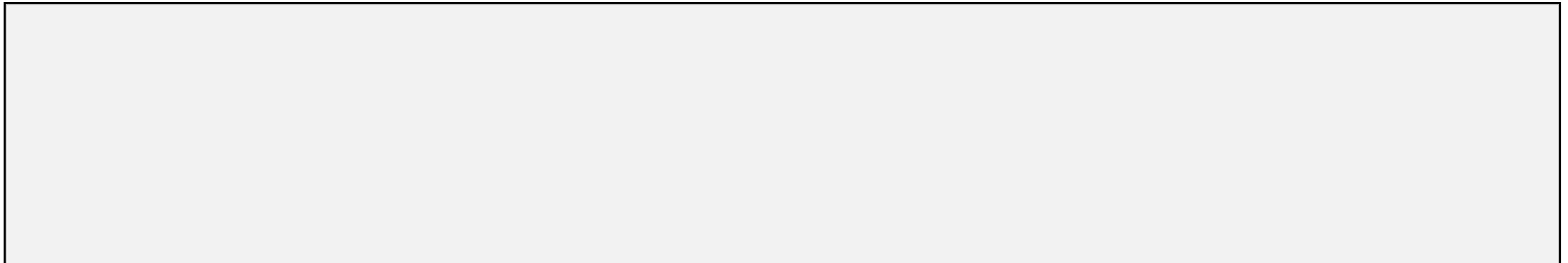
The retrieved context is integrated with the original prompt to provide additional knowledge to the model.

The model processes the enhanced input to produce a final, context-aware response.

For a detailed walkthrough of these concepts, please refer to the previous class recordings on RAG.

RECAP

Retrieval-Augmented Generation (RAG)



For a detailed walkthrough of these concepts, please refer to the previous class recordings on RAG.



Any Questions?

Now's your chance — drop them in the chat or unmute!

Sparse & Dense Vectors



Vectors in Search

We've already seen how vectors are an effective way to organize and store objects and object collections in containers for use in machine learning applications — used in recommendations, classification, and more.

?? Dense Vectors

SEMANTIC SEARCH

Capture meaning and context behind a query. Used in semantic search to find conceptually similar results — even without exact keyword matches.

LEXICAL SEARCH

Capture surface-level term matching. Used in lexical search to find exact keyword matches — fast, predictable, and domain-agnostic.

?? Sparse Vectors

Dense Vectors

Dense vectors are numerical data representations widely used in machine learning and data analysis. They consist of arrays with real numbers, where most or all elements are non-zero.

Key Characteristics

[redacted]

Dimensionality ranges from hundreds to thousands (128, 256, 768, 1024)

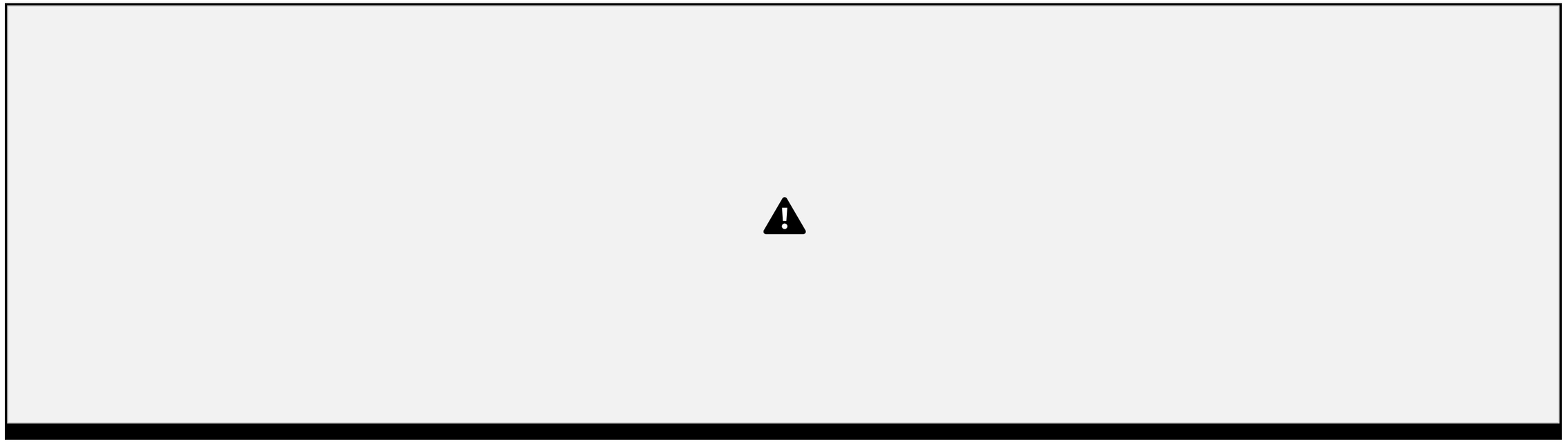
[redacted]

Primary Use Cases

[redacted]

- Dense vectors are generated by embedding models (e.g. BERT, OpenAI text-embedding) and stored in vector databases for fast similarity search.

Dense Vectors



Sparse Vectors

Sparse vectors are an important method of capturing surface-level term matching in information retrieval and natural language processing.

Key Characteristics

High-dimensional vectors where most elements are

zero

phrases Lightweight and fast to compute

Example: [0, 0, 0, 0.8, 0, 0, 0.3, 0, 0, ..., 0]

Typically used with BM25 or TF-IDF algorithms

Why Use Sparse Vectors?

More predictable matching results

Great for special terms, IDs, and exact

Sparse Vector Representation



How Are Sparse Vectors Generated?

⚠ Statistical Techniques

TF-IDF (Term Frequency-Inverse Document Frequency) — assigns weights to words based on frequency and importance across a corpus

BM25 (Best Matching 25) — a ranking function that improves on TF-IDF for information retrieval

⚠ Neural Sparse Embedding Models

Learned methods that generate sparse

representations by training on large datasets —

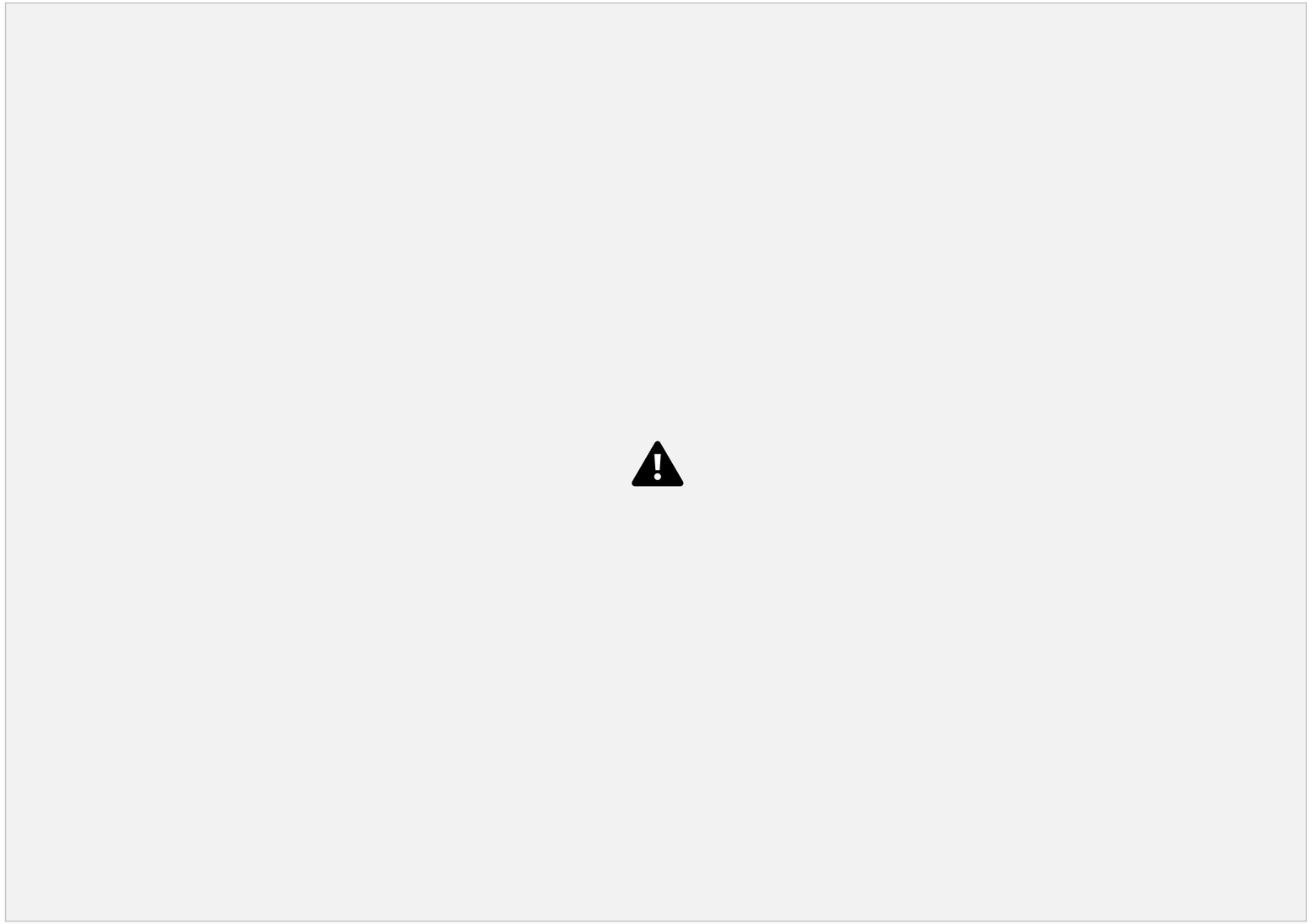
combining the efficiency of sparse vectors with the

intelligence of neural networks.

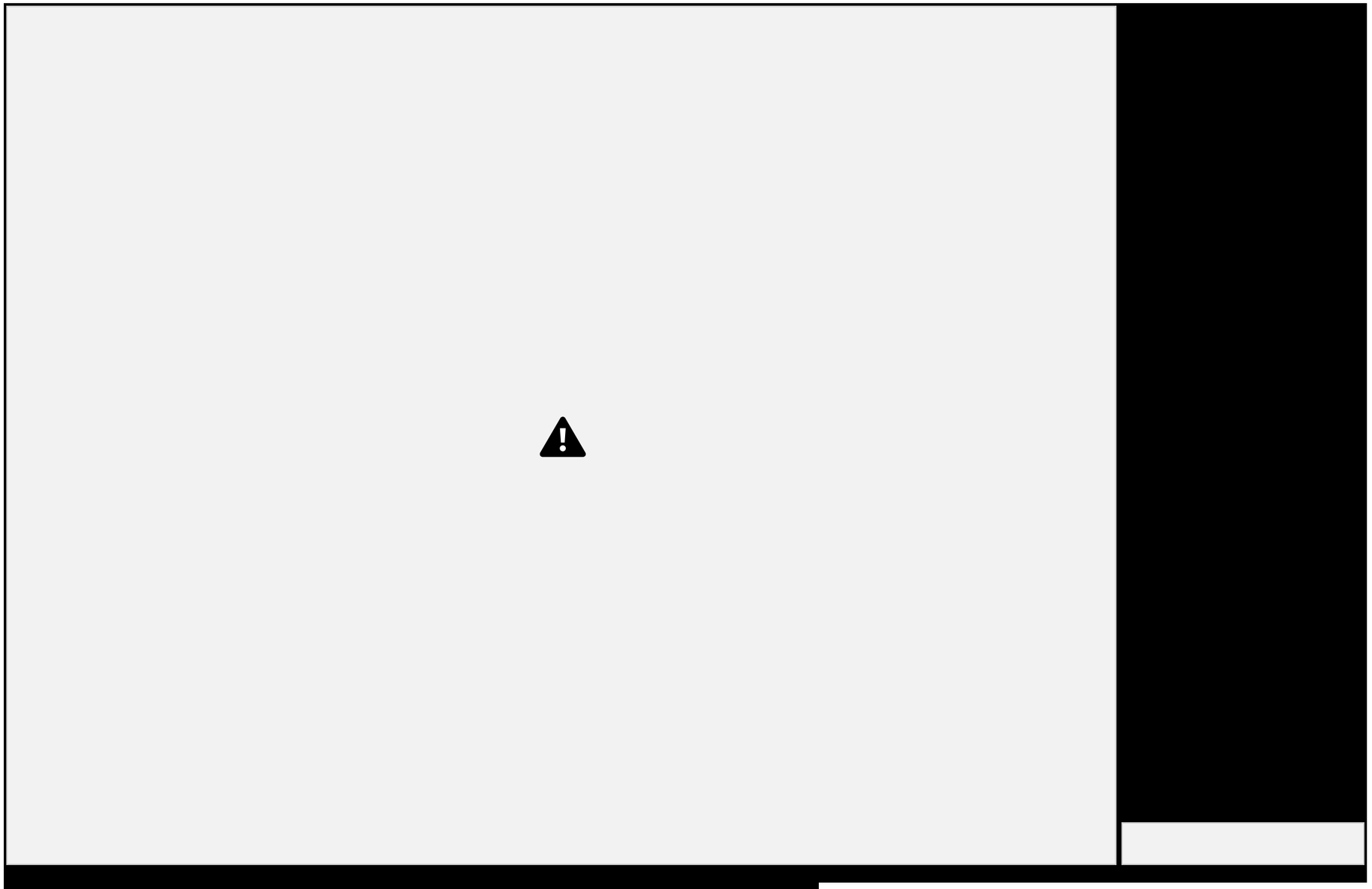
Examples: SPLADE, UniCOIL

We Know How to Convert Text Into Vectors.

Both dense and sparse. Great. But here's the thing...

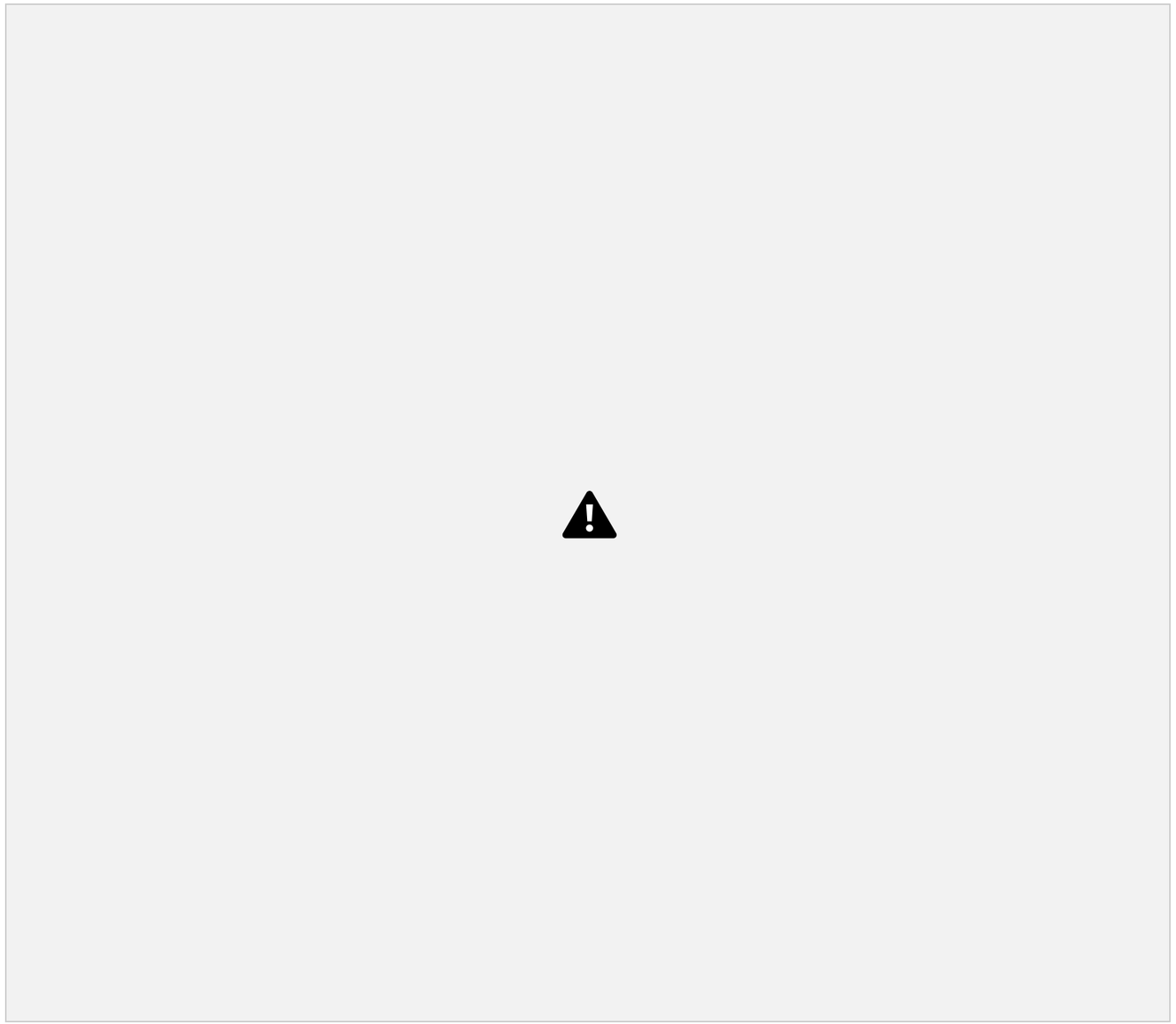


Distance = A Ruler for Meaning



Distance Between Vectors

To find similar vectors, we need a way to measure how close or far apart they are.



Distance Between Vectors

To find similar vectors, we need a way to measure how close or far apart they are.

Distance Between Vectors (cont.)

Euclidean Distance

Measures the straight-line distance between two points in vector space. Best for dense vectors where magnitude matters.

$$\sqrt{\Sigma(a_i - b_i)^2}$$

Best for: image embeddings, spatial data

Cosine Similarity

Measures the angle between two vectors — ignoring magnitude. Best for text embeddings where direction (meaning) matters more than size.

$$(A \cdot B) / (|A| \times |B|)$$

Best for: NLP, semantic search, text embeddings

Manhattan Distance

Measures distance as the sum of absolute differences along each dimension — like navigating city blocks. Robust to outliers.

$$\Sigma|a_i - b_i|$$

Best for: high-dimensional sparse data, robust comparisons

Chebyshev Distance

Measures the maximum absolute difference across any single dimension. Useful when the largest difference in

any one feature dominates.

$$\max(|a_i - b_i|)$$

Best for: grid-based movement, feature-dominant comparisons

For most NLP and semantic search tasks, Cosine Similarity is preferred.

From Distance to Search

Now we know we can use vector embeddings to represent our objects, and the distances between vectors represent the similarity between the objects themselves.

This is where similarity search — or vector search — kicks in.

Given a set of vectors and a query vector, find the most similar items in the set for the query.

The Task

❓ What It's Called

Nearest Neighbor Search — finding the vectors closest in distance to the query vector.

K Nearest Neighbors (k-NN)

K Nearest Neighbors is a popular algorithm to find the nearest vectors in a space for a given query vector. The k is a hyperparameter that defines how many nearest neighbors to retrieve.

⚙ How It Works

Takes a query vector as input

Calculates distance between query and every vector in the database

Returns the k closest vectors based on distance metric

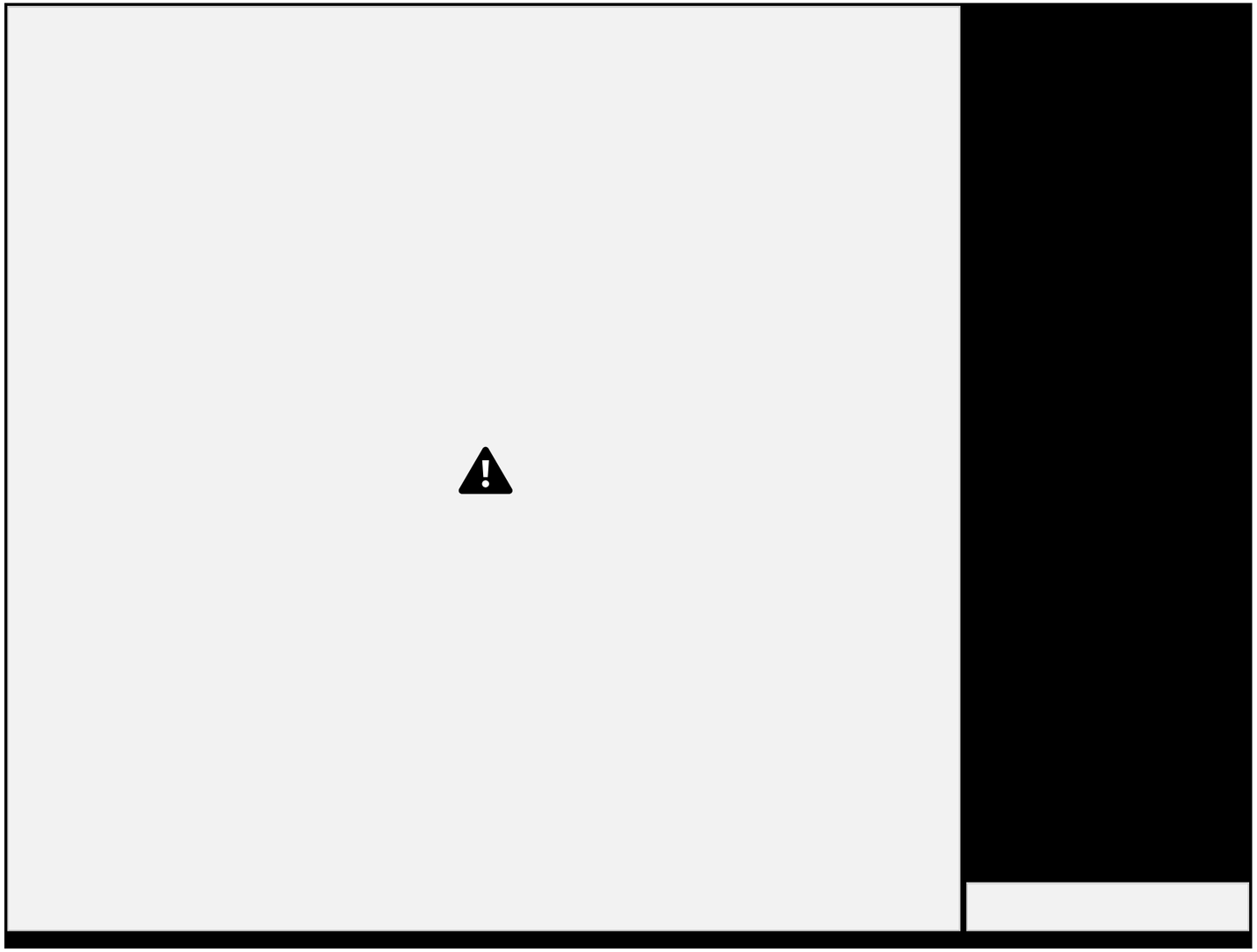
The Major Drawback

To find the nearest vectors, k-NN must calculate the distance between the query and every single vector in the database.

O(n) complexity — scales linearly with data size Extremely slow at millions or billions of

vectors

K Nearest Neighbors (k-NN)



Approximate Nearest Neighbor (ANN)

To reduce the computational complexity of exhaustive search like k-NN, we use Approximate Nearest Neighbor search — trading a small amount of accuracy for massive performance gains.

⚡ The Key Idea

Instead of calculating distances to every vector in the database, ANN retrieves a 'good guess' of the nearest neighbor — enabling search to scale to millions or billions of vectors.

Builds index structures to narrow down search space

Works with the true intrinsic dimensionality of data

Massive performance boost on huge datasets

BEST ANN ALGORITHM

HNSW — Graph-Based Indexing

Hierarchical Navigable Small World (HNSW) indexes a vector space by creating a hierarchical proximity graph — stacking layers of proximity links between vectors.

How It's Built

Draws proximity links (edges) between vectors (vertices) on each layer

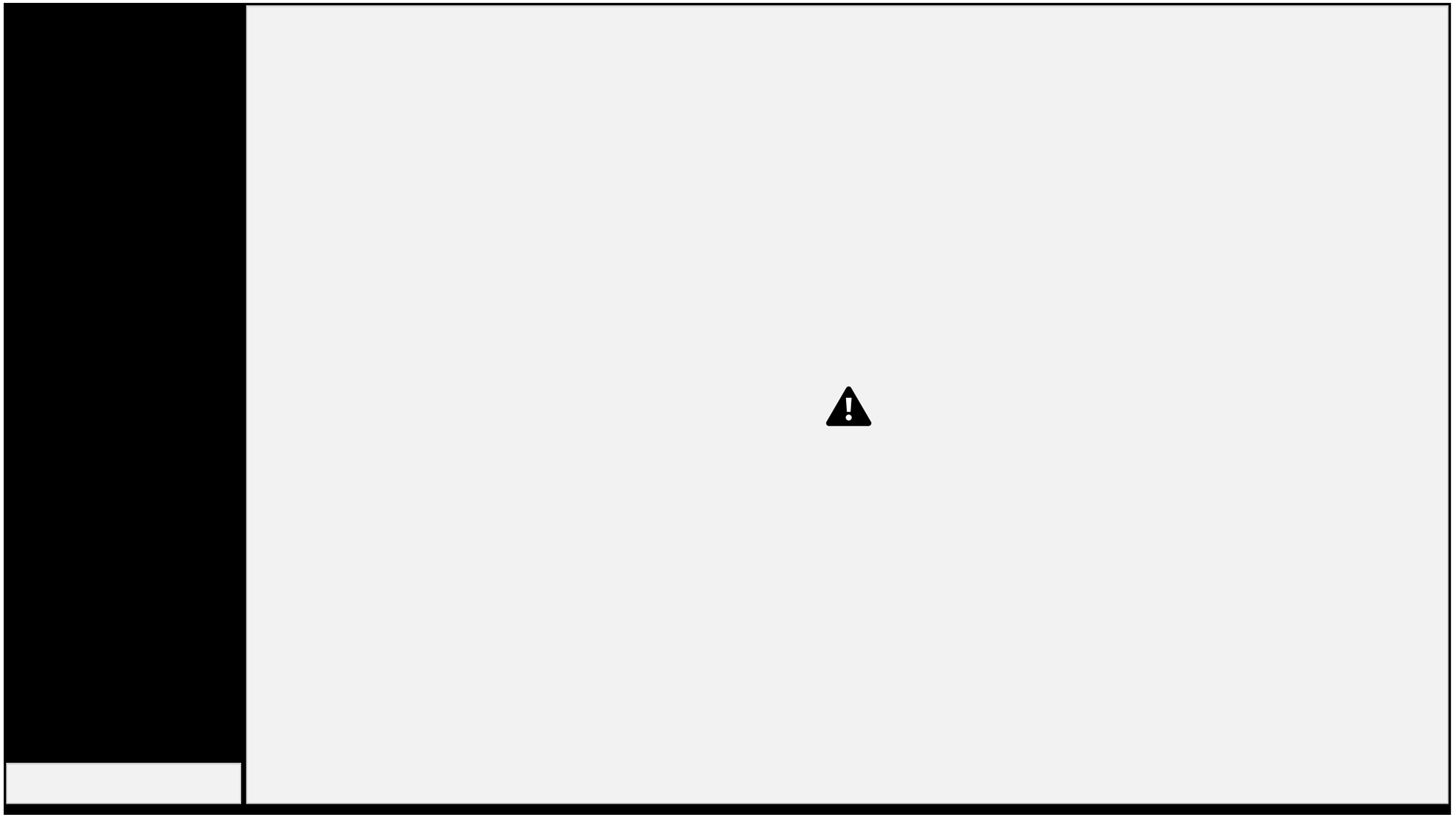
Forms a single-layer proximity graph per level

Stacks layers into a hierarchical graph

Bottom layer holds ALL vectors and their links Upper layers hold progressively fewer vectors

HNSW achieves sub-linear search time by narrowing down candidates layer by layer — making it one of the fastest and most accurate ANN algorithms available. It's the default index in Qdrant.

HNSW — Graph-Based Indexing



BEST ANN ALGORITHM

HNSW — How Search Works

Once the hierarchical proximity graph is built, search traverses it top-down to find the nearest neighbor efficiently.

⚡ The Key Idea



[redacted]Start with a vector as the entry point on the top (sparsest) layer.

Follow Proximity Links

Move gradually to the nearest vector along the available proximity links at that layer.

[redacted]Once the nearest vector is found at the current layer, use it as the entry point on the layer below.

[redacted]

Repeat until the nearest vector is found at the bottom layer — which holds all vectors.

HNSW Solved Dense Search. What About Sparse?

HNSW gives us blazing-fast approximate nearest neighbor search for dense vectors. But sparse vectors are fundamentally different — most dimensions are zero, and they need a different kind of index.

✅ Dense Search — Solved

HNSW graph-based index navigates high-dimensional dense vectors efficiently.

Used for semantic / vector search

Sparse Search — The Inverted Index

Unlike dense vectors which use HNSW, sparse vectors are indexed using an inverted index — the same structure powering traditional search engines like Elasticsearch.

?? How It Works

An inverted index maps each unique term to a list of documents (and their weights) that contain that term — making exact term lookups near-instant.

Each dimension = a vocabulary term Only non-zero dimensions are stored Lookup is O(1) per

term — extremely fast No need to scan all vectors

Sparse Search — The Inverted Index

Let's consider a small example: we have a collection of six documents containing the terms 'Cat', 'Dog', and 'Fish'. Each term is mapped to a list of document IDs where it appears.

Term Document ID List

- Cat [1, 3, 6]
- Dog [2, 5]
- Fish [4]

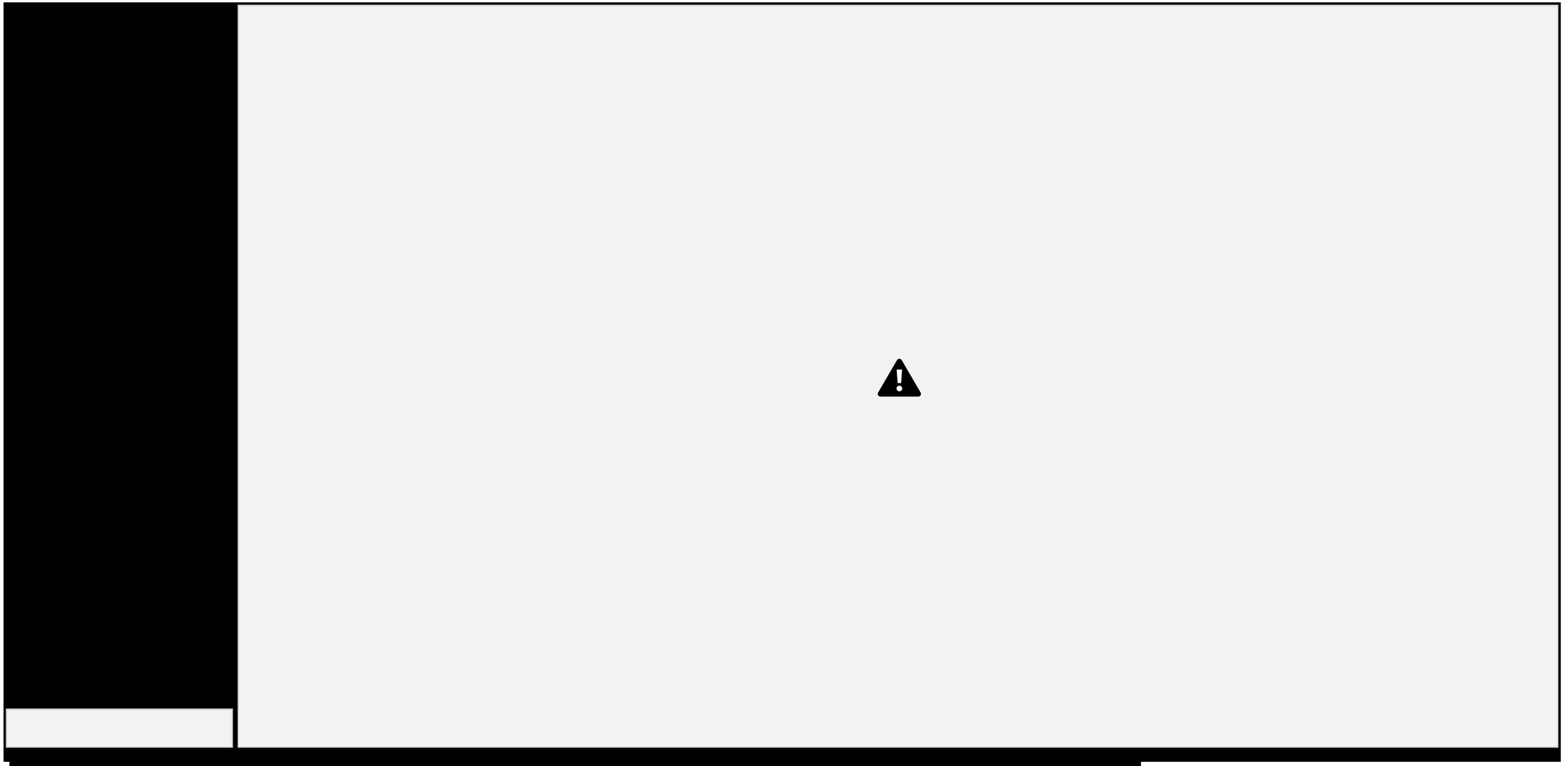
?? Query: 'Cat'

The index instantly returns documents [1, 3, 6] — no need to scan all documents.

?? Query: 'Cat AND Dog'

Intersect the lists: [1, 3, 6] ∩ [2, 5] = [] — no documents contain both terms.

Sparse Search — The Inverted Index



Dense vs Sparse Index — Side by Side

Dense Index(HNSW)

SEMANTIC SEARCH

Graph-based approximate nearest neighbor All dimensions are

non-zero

Searches by proximity in vector space Best for: embedding

model outputs Trade-off: approximate, uses more memory

Sparse Index (Inverted Index)

LEXICAL SEARCH

Term-to-document mapping structure

Most dimensions are zero — only non-zero stored Searches by

exact term lookup

Best for: SPLADE, BM25, TF-IDF outputs

Trade-off: exact but limited to term matching

Any Questions?

Now's your chance — drop them in the chat or unmute!

Break Time!

Take a 10-minute pause to recharge.



Enjoy a Warm Drink

Sip on your favorite beverage.

Set Your Timer

You'll be refreshed in 10 minutes.

Find Your Zen

Breathe and clear your mind.

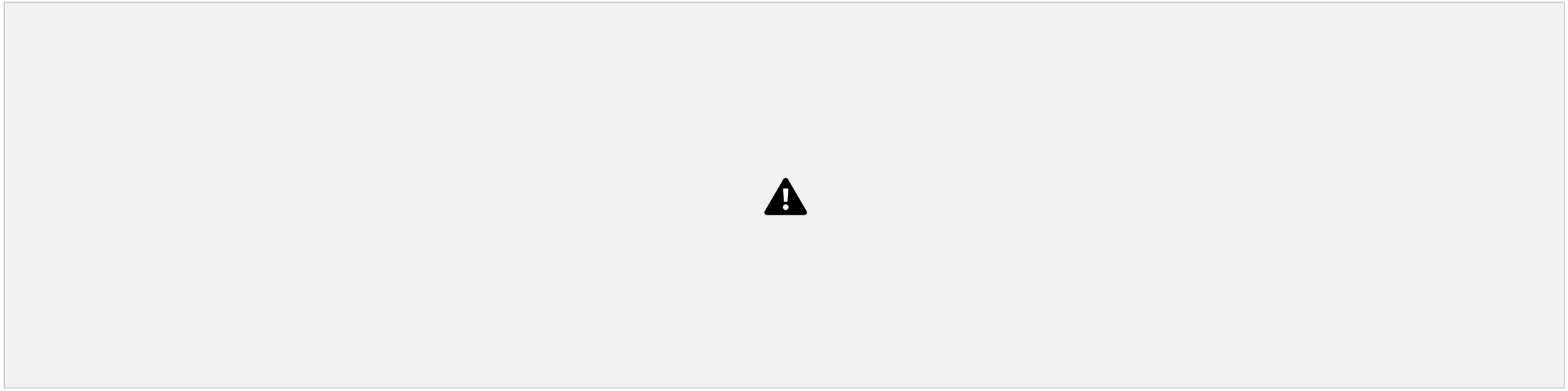
Evolution of Lexical Search

From Bag of Words to SPLADE — how sparse retrieval learned to understand meaning.

Evolution of Lexical Search

Term Matching in Conventional Lexical Search

Algorithms like BM25 and TF-IDF rely on exact term matching — they look for the precise words in a query within the documents. If the words don't match, the document won't be retrieved.



Evolution of Lexical Search

Problem with Conventional Algorithms

Despite having a query and document with many relevant terms, because they are not “exact matches” they are not identified.

The Solution

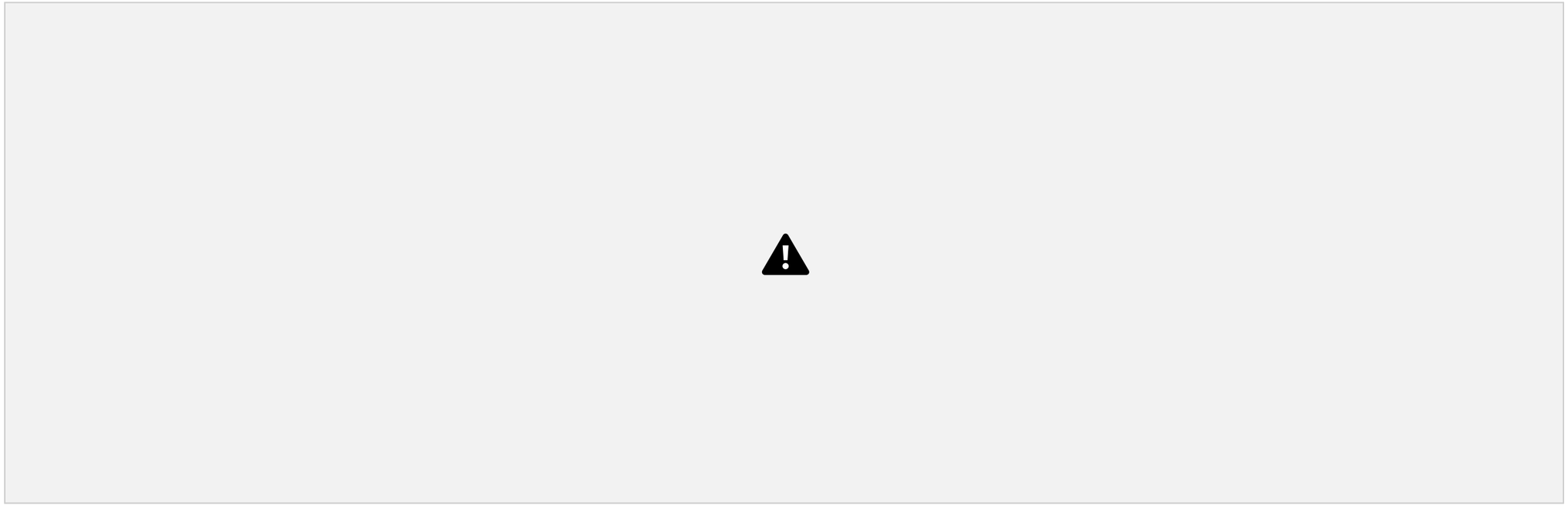
Term expansion is crucial in minimizing the **vocabulary mismatch problem** — the typical lack of term

overlap between queries and relevant documents.

Evolution of Lexical Search

With term expansion on our query we will see a much larger overlap because we’re now able to identify similar words.

Term expansion allows us to identify relevant but different terms and use them in the sparse vector retrieval step



That's Where SPLADE Comes In

Learnable sparse embedding models help minimize the traditional drawbacks of sparse retrieval — bridging the gap between exact term matching and semantic understanding.

The Long Road to Learned Sparsity



Traditional methods like TF-IDF and BM25 transform documents into sparse frequency vectors. Hugely popular for their efficiency, interpretability, and exact term matching — but far from perfect.

Humans rarely know the exact terms contained in the documents

they're looking for.

2. Dense Embedding Models — A Step Forward

Dense models enable search based on semantic meaning rather than term matching. But they require vast amounts of data to fine-tune — without it, they underperform sparse methods in niche domains.

Problematic for specialized domains where data is scarce and terminology is specific.

3. ⚡ SPLADE — Learnable Sparse Embeddings

Learnable sparse embedding models help minimize the traditional drawbacks of sparse retrieval. One of the most performant models in this space is SPLADE — combining the efficiency of sparse vectors with the learned semantic understanding of neural models.

SPLADE bridges the gap between sparse and dense retrieval — giving us robust, learnable sparse embeddings that work well even in niche domains.

Sparse Lexical and Expansion

Pretrained language models like BERT can identify connections between words and use that knowledge to enhance sparse vector embeddings — in two powerful ways.

⚖ Term Weighting

Weighs the relevance of different terms intelligently.

Common words like 'the' or 'will' carry less relevance, while

rare or domain-specific words like 'orangutan' are weighted higher.

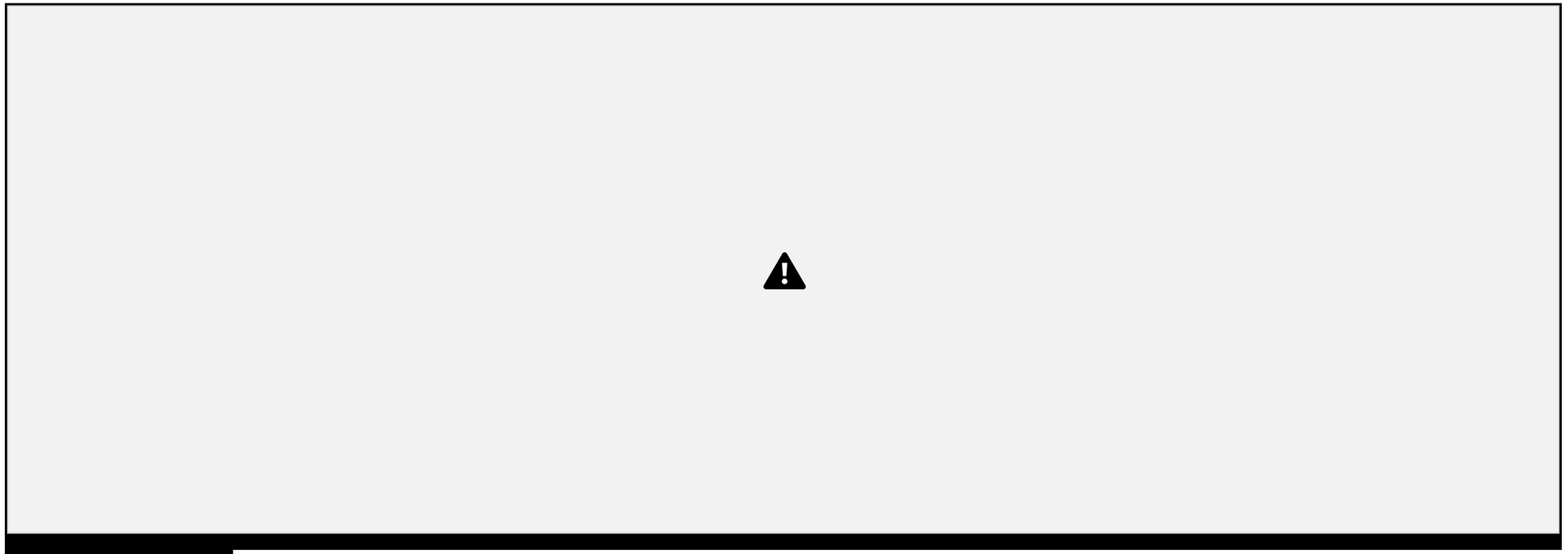
found in the original sequence — enriching the sparse vector with semantically related vocabulary the original text didn't explicitly contain.

Term Expansion

Includes alternative but relevant terms beyond those

Together, term weighting and expansion give SPLADE the precision of sparse retrieval with the semantic awareness of dense models — without needing a full dense vector.

Sparse Lexical and Expansion



Benefits of Using SPLADE

SPLADE has proven to be a great approach to handling sparse vectors and offers several benefits over its counterparts.



[redacted]

Any Questions?

Now's your chance — drop them in the chat or unmute!

Break Time!

Take a 10-minute pause to recharge.



Enjoy a Warm Drink

Sip on your favorite beverage.

Set Your Timer

You'll be refreshed in 10 minutes.

Find Your Zen

Breathe and clear your mind.

Hybrid Search

Combining the best of lexical and semantic search.

Hybrid Search

Hybrid search is an information retrieval technique that blends two or more search methods (e.g., lexical search and semantic search) into a single ranked list to improve relevance and recall.

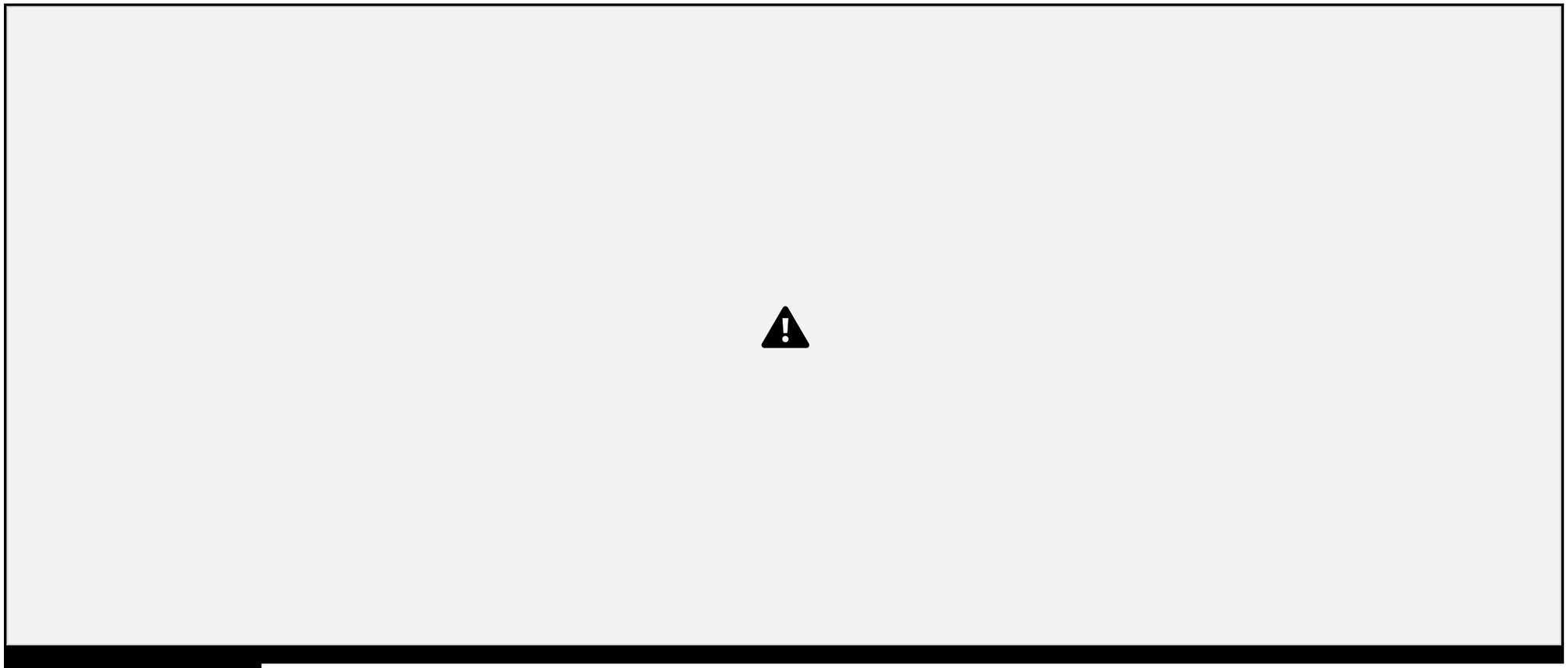
?? Lexical Search

Matches exact words and phrases — great for precision.

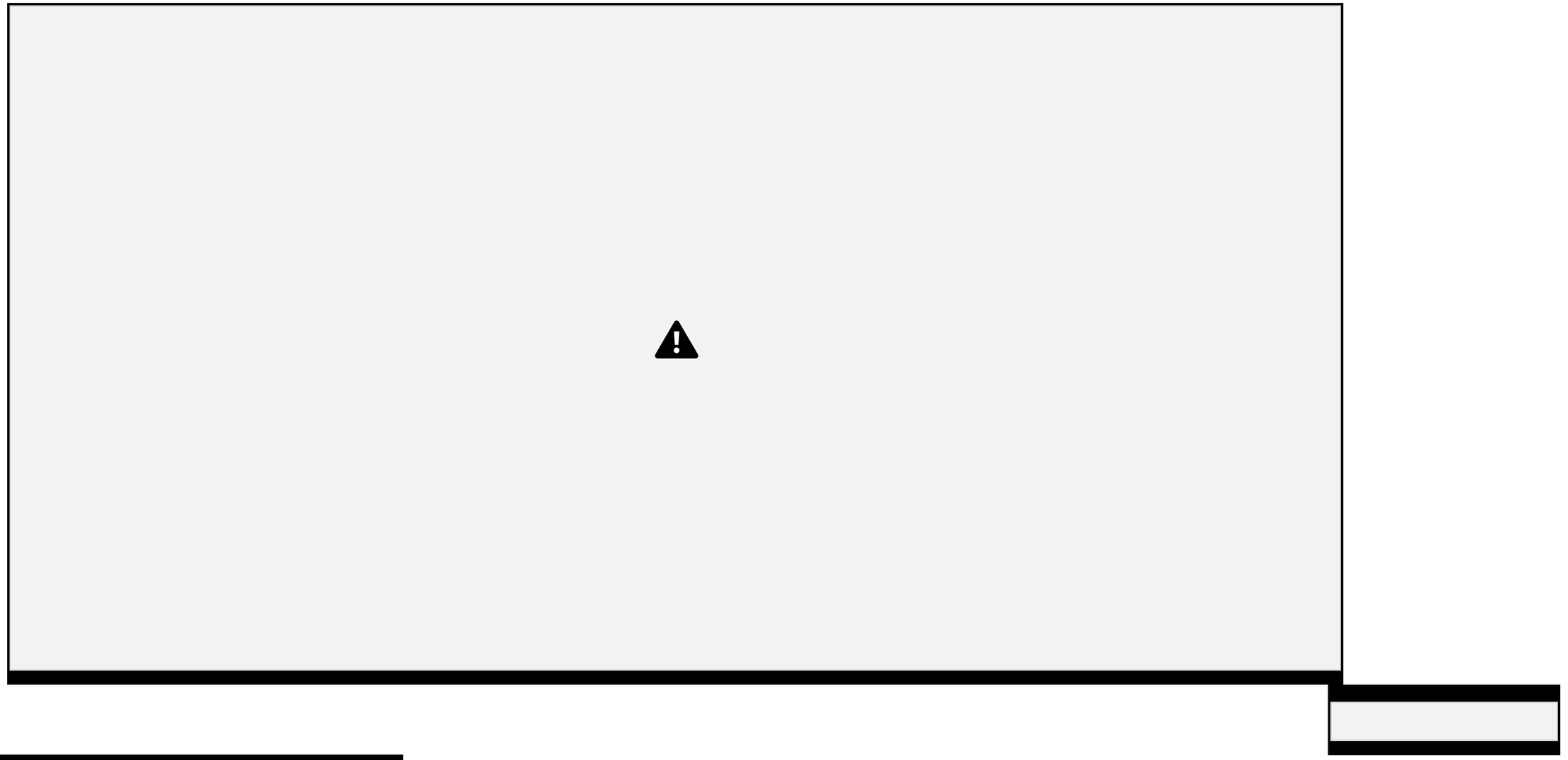
?? Semantic Search

Interprets the meaning behind a query — great for understanding intent.

Hybrid Search — Indexing



Hybrid Search — Querying



Hybrid Search in

Action Query: "Italian recipes with tomato sauce"

Keyword Search

Matches exact terms

• Italian Tomato Sauce Pasta • Italian Recipes with Tomato Sauce

• Tomato Sauce for Italian Dishes

Misses: 'Spaghetti Marinara', 'pasta sauce' variations

Semantic Search

Understands intent & context

• Spaghetti Marinara (traditional) • Pasta al Pomodoro

Mexican Salsa Verde

Over-broad: surfaces unrelated tomato-based dishes

Hybrid Search

Best of both worlds

• Spaghetti Marinara

• Pasta al Pomodoro

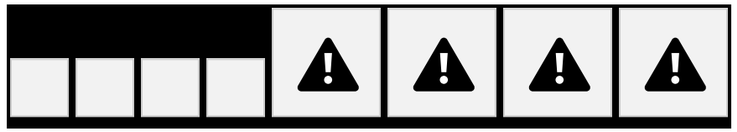
• Classic Italian Tomato Sauce • Penne Arrabbiata

Precise AND contextually relevant

Why Hybrid Search Wins

either approach alone

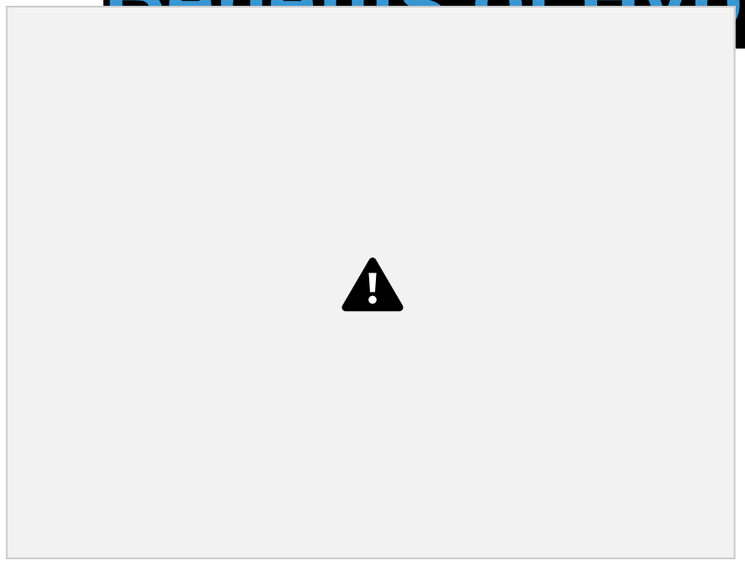
Better relevance than



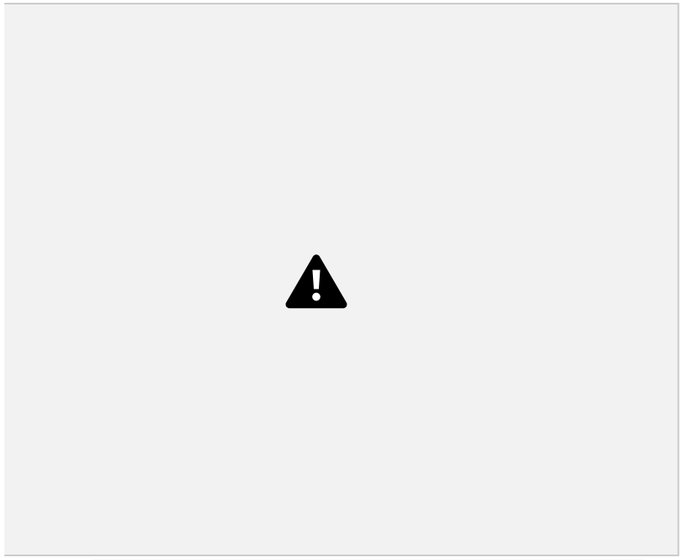
Handles both exact and fuzzy queries

Builds user trust with consistent results

Benefits of Hybrid Search



Combines exact matches with semantic understanding — delivering high-quality results that retain users and reduce bounce rates.



Enhanced Accuracy & Relevance

Delivers meaningful results even

with vague or inaccurate queries. Users find what they need faster, enabling more engaging search experiences.

Combining fast keyword lookup with semantic search optimizes both quality and speed — improving contextual accuracy without sacrificing performance.

Systems can dynamically adjust the weight of keyword vs. semantic relevance — or give users direct control over it for a tailored search experience.

When to Consider Hybrid Search

The decision

depends on what your users are looking for. Here's a quick guide: ⚠ ⚠ ⚠

❓❓ [redacted]

Developers need exact lines of code or error messages

Recommendation: Use Keyword Search — specific term matching is ideal

❓❓ [redacted]

Users search for advice or experiences related to their feelings

Recommendation: Use Semantic Search — meaning matters more than exact words

[redacted]

Customers search for specific products but are open to related suggestions

Recommendation: Use Hybrid Search — exact matches + contextual discovery

Ask yourself: Do my users search with exact terms, fuzzy intent, or both? That answer points you to the right approach.

Where Hybrid Search

Shines

[redacted]

Improves retrieval quality by combining exact chunk matching with semantic relevance.

[redacted]

Handles brand names, SKUs, and fuzzy intent like 'comfortable' or 'affordable' simultaneously.

[redacted]

Finds policy docs by keyword while surfacing conceptually related content across departments.

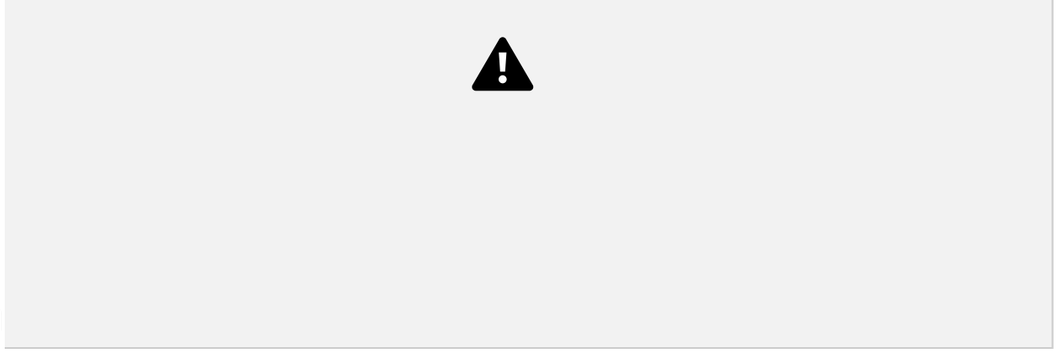
[redacted]

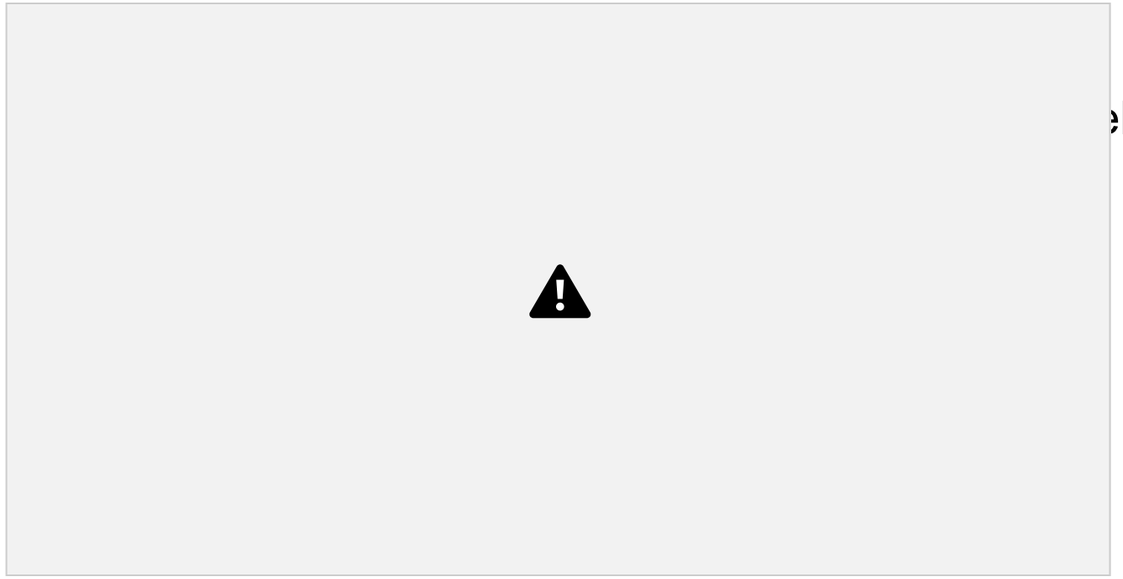
Matches ticket IDs and product names while understanding the user's underlying issue.

Drawbacks of Hybrid Search

[redacted]

Requires integrating multiple search algorithms





lying on

benefits of the other. Without the right balance between keyword precision and semantic power, result quality suffers — leading to poor user experience and higher bounce rates.

(e.g., dense embeddings + keyword search). Deep technical understanding is needed — it's not a plug-and-play solution.

Any Questions?

Now's your chance — drop them in the chat or unmute!

How to Combine Search Methods

Hybrid search merges keyword search and semantic search — but how does this process work?

- [redacted]

  Keyword search finds results matching specific words or phrases. Semantic search finds results based on context and meaning — each producing its own independent result list.

- [redacted]

  The two result lists are merged into a single unified list through a process called fusion — combining results based on a ranking or scoring system.

- [redacted]

The fused list integrates the strengths of both methods — prioritizing results by relevance, individual rankings, and other criteria to deliver the most comprehensive results.

FUSION STRATEGY

Reciprocal Rank Fusion (RRF)

The key to hybrid search is the fusion step — how you merge and rank the two result sets determines the quality of the final output.

This is where RRF (Reciprocal Rank Fusion) comes in.

RRF is a rank aggregation method that combines multiple ranked lists into a single, unified ranking — without needing relevance scores from each individual search method. It works by assigning each result a score based on its position (rank) in each list, then summing those scores to produce a final ranking.

Originally proposed by Cormack, Clarke & Buettcher (2009) — now widely used in hybrid search pipelines.

How RRF Works

RRF gives more weight to top-ranked items in each individual result list when building the final combined list.

Looks Over Each Record Each record is assigned a score — it may exist in one or both result lists (keyword + semantic).

Calculates the Score Score = sum of 1/rank across all lists. Example: ranked 3rd in keyword + 9th in semantic = 1/3 + 1/9 = 0.444. If a record appears in only one list, it scores 0 for the other.

Sorts the Results Records are sorted by final score — highest scores ranked first. Items ranked high in multiple lists rise to the top; items ranked high in only one list are penalized.

RRF — Smoothing Constant k

To prevent extremely high scores for top-ranked items, a constant k is added to the denominator:

```
Score = 1 / (k + rank)
```

Without k

A record ranked 1st scores: 1/1 = 1.0

Extreme score — dominates the final ranking

With k = 60

A record ranked 1st scores: 1/(60 +1) = 0.016

Balanced — reduces dominance of top-ranked items

Studies have shown that k = 60 performs well across various datasets and retrieval tasks.

Placing rank in the denominator naturally penalizes low-ranking records, while k smooths out extreme scores at the top — giving a fairer, more balanced final ranking.

Any Questions?

Now's your chance — drop them in the chat or unmute!

Break Time!

Take a 10-minute pause to recharge.



Enjoy a Warm Drink

Sip on your favorite beverage.

Set Your Timer

You'll be refreshed in 10 minutes.

Find Your Zen

Breathe and clear your mind.



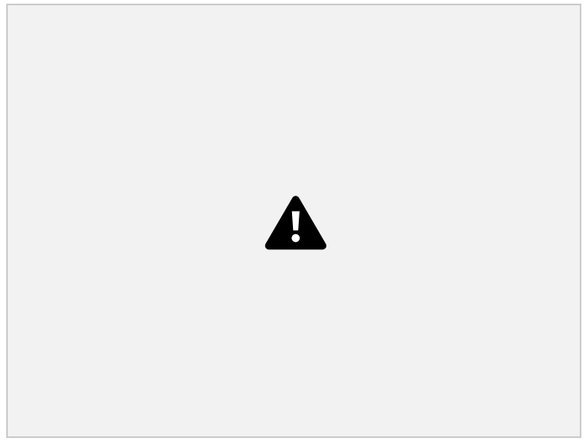
Before We Jump Into Code

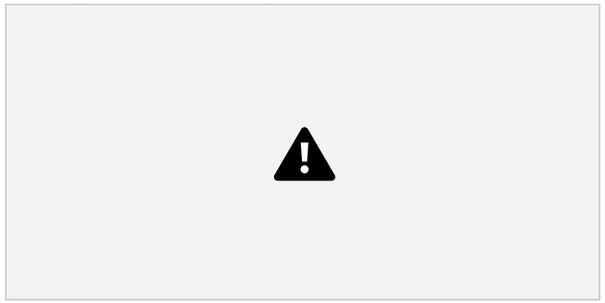
Let's see Tools & Libraries in focus

Tools & Libraries In Focus

Before jumping into code, let's get familiar with the tools and libraries we'll be working with today.







Qdrant

Open-source vector search engine supporting

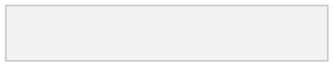
both dense and sparse vectors with built-in

hybrid search.

Hugging Face

Provides pretrained embedding models

including SPLADE for sparse vector generation.



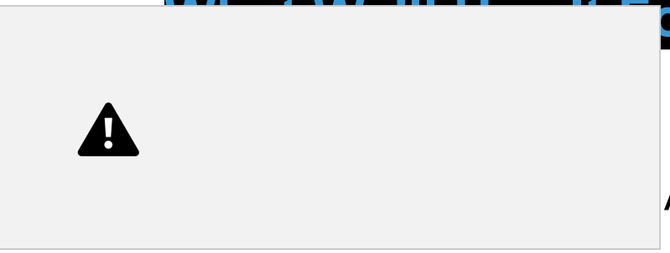
FastEmbed

Lightweight Python library for generating fast, accurate, state-of-the-art embeddings.

Qdrant

Qdrant is an open-source vector similarity search engine used to store, organize, and query collections of high-dimensional vectors.

What We'll Use It For



Add a collection of documents to the engine Query the collection to retrieve the most relevant documents



Implement hybrid search with sparse and dense vectors



Apply RRF fusion and reranking

Qdrant Key Features

[redacted]

Vector similarity search optimized for speed and scale.



[redacted]

Natively supports both dense and sparse vectors for hybrid search.

[redacted]

Built-in metadata storage and filtering for fine-grained queries.



[redacted] Designed to scale horizontally with growing data volumes.

[redacted]

Fully open-source with a managed cloud option for easy deployment.

Qdrant Data Structure

[redacted]



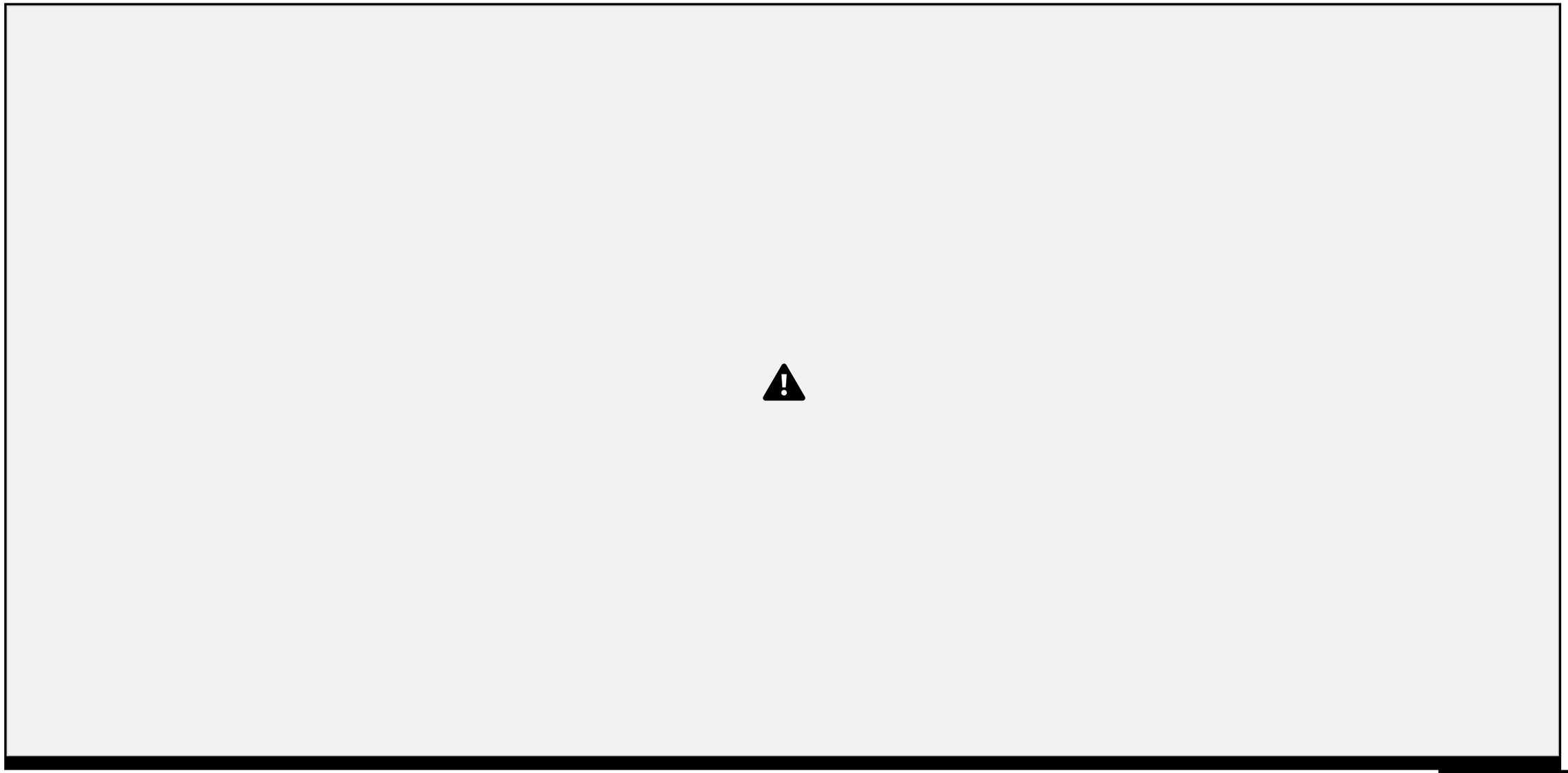
The top-level container — similar to a table in a relational database. Each collection stores points and defines the vector configuration (size, distance metric).

The core data unit in Qdrant. Each point consists of:

- A unique ID
- One or more vectors (dense and/or sparse)
- An optional payload (metadata)
  Key-value metadata attached to each point — used for filtering, faceting, and enriching search results. Supports strings, numbers, booleans, and nested objects.

In hybrid search, each point stores both a dense vector (for semantic search) and a sparse vector (for lexical search) — queried in parallel at search time.

Qdrant Data Structure



FastEmbed

FastEmbed is a lightweight, fast Python library built for embedding generation — and it's our tool of choice for generating sparse vectors with SPLADE.

⚡ **Native SPLADE Support**

Includes sparse embedding models like SPLADE out of the box.

🔽 **No PyTorch Dependency**

Uses quantized model weights and ONNX Runtime.

🔗 **Ecosystem Integration**

Integrates natively with LangChain, LlamaIndex, etc.

🚀 Runs on existing infrastructure.

Why Do Libraries Like FastEmbed Exist?

Working directly with models requires heavy dependencies and boilerplate. FastEmbed abstracts all of that away.

Concern	Direct Model	FastEmbed

Dependencies	PyTorch (~2GB), CUDA optional	ONNX Runtime (~50MB), no PyTorch
Speed	Normal PyTorch inference	Quantized + ONNX optimized, faster on CPU
Deployment	Needs PyTorch in production	Lightweight, serverless-friendly
Fine-tuning	Yes, full control	No, inference only

Any Questions?

Now's your chance — drop them in the chat or unmute!

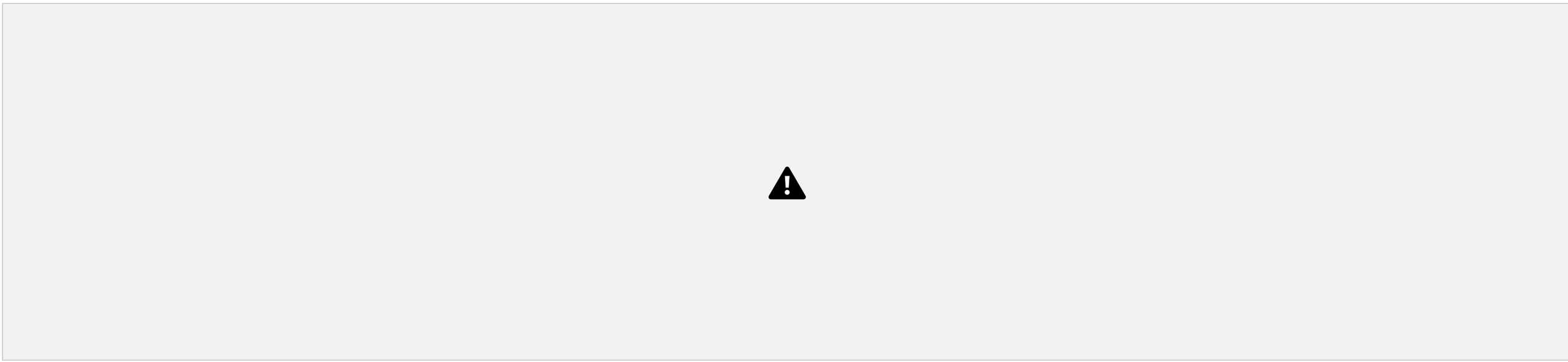


Let's Get Your Hands Dirty with Code

Time to build! Let's put everything we've covered into practice — sparse vectors, dense vectors, hybrid search, RRF, and reranking — all in code.

Any Questions?

Now's your chance — drop them in the chat or unmute!



APPRECIATION GRATITUDE

Thank You!

Questions? Let's Discuss!